# Wearable Gesture Controller: An IMU-Based Edge Device for Gesture-Driven Smart Home Control

Arya Choudhary (240911438), Sanika Singh (240911664), and Vishad Sharma (240911580)
School of Computer Engineering, Manipal Institute of Technology, Manipal, India
ICT 3132 Embedded Systems, V Semester B.Tech. (IT)

***Abstract*—Wrist gestures offer an additional interaction channel for smart-home users with limited speech, reach, or fine motor control. This work presents an inertial measurement unit (IMU)-based gesture-to-command architecture and a simulation-based software evaluation. The pipeline processes six-channel acceleration and angular-velocity signals through causal filtering, normalization, fixed-length windowing, and a compact gated recurrent unit (GRU). A background class and confidence threshold suppress unintended command output. Independent signal sequences are partitioned before windowing, and a validation set selects the model checkpoint. The 16-unit GRU contains 1220 parameters and achieves 100.0% accuracy with a macro-F1 score of 1.000 on 200 held-out simulation windows. These results demonstrate end-to-end operation under the specified signal-generation conditions. An analytical comparison with an LSTM reference and a resource assessment address buffering, parameter storage, recurrent runtime support, and prospective 8-bit quantization. The architecture separates the evaluated software pipeline from sensor acquisition and wireless actuation interfaces, providing a foundation for subsequent embedded deployment and user evaluation.**

***Index Terms*—embedded systems, IMU, gesture recognition, GRU, simulation, edge inference, accessibility**

## I. Introduction and Problem Statement

Smart-home interaction commonly relies on switches, touchscreens, remote controls, or voice assistants. These interfaces assume particular abilities in reach, speech, or fine motor control. A motion-based interface could provide an additional option for users who can perform comfortable, repeatable wrist movements. It would complement existing accessible controls rather than meet every user's needs. Accessibility benefit must ultimately be established with users, not inferred from classification scores.

An IMU supplies three accelerometer and three gyroscope channels without a camera's dependence on lighting or line of sight. A local classifier could translate a short motion sequence into a symbolic command without transmitting raw sensor traces to a cloud service. However, everyday movements can resemble intentional gestures. A useful interface therefore requires both recognition and a reliable no-command path, together with limits on processing delay and resource use.

The proposed controller combines temporal gesture recognition with a compact command interface. The problem addressed here is: *how can a bounded six-channel processing pipeline translate motion input into commands, reject background or uncertain predictions, and accommodate the resource constraints of an embedded system?* A software implementation provides a reproducible environment for investigating this pipeline, while the architecture defines interfaces for subsequent sensor acquisition and appliance integration.

Controlled input streams support reproducible checks of preprocessing, sequence segmentation, classification, and command mapping. The study evaluates these components together and analyses the memory, computation, and runtime requirements relevant to a future embedded implementation.

## II. Focused Literature Survey

Recent work on wearable activity recognition and resource-constrained machine learning provides the foundation for the proposed controller. The following review examines ten studies published in 2020–2022, focusing on temporal representations, multichannel sensor processing, efficient deployment, and embedded evaluation.

### A. Wearable and Inertial Recognition

Zhou *et al.*'s TinyHAR combines local temporal processing, cross-channel interaction, and global temporal modelling to obtain a lightweight activity-recognition model [1]. Abedin *et al.* use cross-channel interaction and temporal attention to improve wearable activity representations in Attend and Discriminate [2]. These approaches motivate preserving both temporal and multichannel information. The proposed controller extends this representation-oriented perspective with an explicit gesture-to-command decision layer.

Saha *et al.*'s TinyOdom addresses inertial navigation with hardware-aware model design and quantization-aware search [3]. It highlights the importance of sensor disturbances and target-aware resource constraints in inertial applications. Whereas TinyOdom uses temporal convolutional models for navigation, the present system uses a compact recurrent model to classify command gestures.

### B. Resource-Aware Embedded Learning

MCUNet jointly designs network architecture and an inference engine for resource-limited IoT devices [4]. MCUNetV2 introduces patch-based inference to address peak activation memory [5]. DORY explicitly manages tiling and data transfers when deploying neural networks on low-cost MCUs [6]. These studies establish that deployment feasibility depends on activation lifetimes, buffers, operator implementations, and the memory hierarchy as well as parameter count. Recurrent

TABLE I
LITERATURE THEMES AND THEIR IMPLICATIONS FOR THIS PROJECT

| Reviewed work | Implication for the proposed controller |
|---|---|
| TinyHAR; Attend and Discriminate [1], [2] | Temporal and cross-channel representations motivate a compact sequence classifier with a command decision layer. |
| TinyOdom [3] | Inertial robustness and hardware-aware design guide sensor conditioning and resource analysis. |
| MCUNet; MCUNetV2; DORY [4]–[6] | Activation storage, data movement, and recurrent-kernel support inform deployment feasibility. |
| Once-for-All; TinyTL; constrained training [7]–[9] | Task-specific model sizing and offline training simplify the controller's runtime requirements. |
| MLPerf Tiny [10] | Classification quality, latency, memory, and energy define distinct evaluation dimensions. |

inference therefore requires a corresponding analysis of kernel and state-storage support.

Once-for-All specializes a trained network for deployment constraints [7]. TinyTL reduces activation-memory demands for on-device adaptation [8], while On-Device Training Under 256KB Memory combines algorithm and runtime design for constrained training [9]. These studies motivate task- and target-specific decisions. The proposed controller uses offline host training and a fixed compact architecture, allowing inference requirements to be analysed independently of on-device adaptation.

MLPerf Tiny treats embedded inference quality, latency, and energy as separate benchmarking dimensions [10]. This distinction guides the evaluation of classification performance, host inference time, parameter storage, and prospective on-target requirements. Table I summarizes the reviewed themes and their implications for the controller architecture.

## III. RESEARCH GAP, OBJECTIVES, AND CONTRIBUTIONS

The reviewed studies address complementary parts of the design problem. Wearable recognition models emphasize temporal and cross-channel representations, whereas deployment frameworks emphasize resource scheduling and target-specific execution. A command interface additionally requires an explicit decision layer that distinguishes an intentional action from background or uncertain input. The application-specific gap addressed here is the integration of these concerns in a bounded, reproducible sensor-to-command pipeline: six-channel preprocessing, fixed-size windows, a compact temporal classifier, and command rejection.

The objectives are to: (1) implement a repeatable six-channel input-to-command pipeline; (2) maintain independent training, validation, and test partitions; (3) incorporate a no-command decision for background and low-confidence input; and (4) analyse model size, buffering, and inference requirements for embedded execution. The contributions are an integrated software implementation, a reproducible evaluation protocol, an analytical LSTM–GRU resource comparison, and a platform-independent deployment architecture.

## IV. PROPOSED METHODOLOGY AND ARCHITECTURE

Fig. 1 presents the processing architecture and its physical integration interfaces. The software processes six channels, $\mathbf{x}_t = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^\mathsf{T}$. For embedded acquisition, an MCU would read an IMU through I$^2$C or SPI using a timer or data-ready interrupt. Timestamping, missed-sample detection, and saturation checks would maintain a consistent input stream. The platform-independent interfaces separate these acquisition functions from the classification and decision stages.

### A. Acquisition, Filtering, and Windowing

The processing settings are a nominal $50$ Hz sample rate, $64$ samples per window, and a stride of $32$ samples. A window represents $1.28$ s of signal, with a new classification opportunity every $0.64$ s. Window duration and stride determine observation and update intervals; processing and communication contribute additional response delay. An embedded fixed-size ring buffer would retain the most recent samples, while the software implementation constructs equivalent windows from finite arrays.

A causal first-order filter is independently reset at each recording boundary:

$$\mathbf{y}_0 = \mathbf{x}_0, \qquad \mathbf{y}_t = 0.35\mathbf{x}_t + 0.65\mathbf{y}_{t-1}. \tag{1}$$

The filter smooths short-term fluctuations while preserving causal execution. Sensor-side anti-alias filtering remains part of the acquisition design. Each channel is standardized using means and standard deviations computed from *unique filtered training samples only*. These constants are frozen for validation, testing, and saved-model inference. The model receives the complete $64 \times 6$ sequence and learns temporal features within its recurrent layer.

### B. Compact GRU and Analytical LSTM Reference

The classifier is a single-layer, unidirectional GRU with $16$ hidden units and a linear four-class output head. A softmax converts logits to class probabilities. The vocabulary contains background, swipe left, swipe right, and wave, mapped to `NO_COMMAND`, `LIGHT_OFF`, `LIGHT_ON`, and `FAN_TOGGLE`, respectively. This compact vocabulary supports directional and oscillatory motion commands. A hold gesture is excluded because stillness requires an explicit interaction context to distinguish it from ordinary inactivity.

Each window starts from a zero recurrent state and is classified from its final hidden vector. This gives bounded, independent inference calls, though overlapping windows recompute some samples. The host implementation uses PyTorch's GRU convention with separate input and recurrent bias vectors. With $d = 6$, $h = 16$, and $C = 4$, its parameter count is

$$P_{\mathrm{GRU}} = 3h(d + h + 2) + (h + 1)C. \tag{2}$$

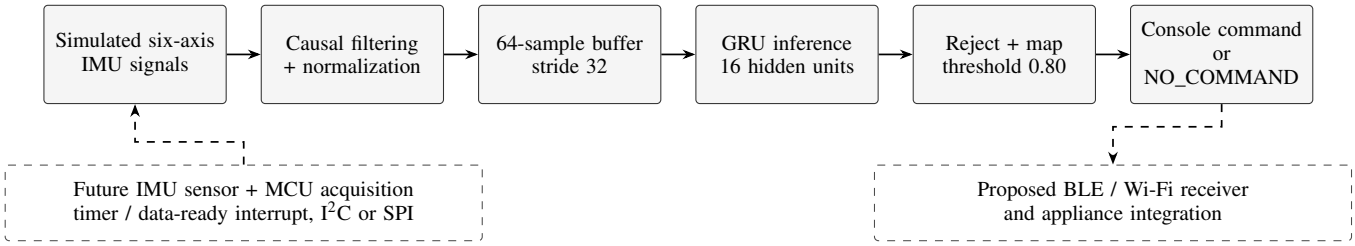



Fig. 1. Sensor-to-command architecture. Solid blocks form the software evaluation path using simulation-generated IMU signals; dashed blocks define the proposed embedded acquisition and appliance integration interfaces.

This gives 1220 parameters. A 32-unit LSTM reference would contain $4h(d+h+2)+(h+1)C = 5252$ parameters under the same double-bias convention. The analytical comparison therefore indicates a $76.8\%$ reduction in parameter count for the selected GRU. This reduction reflects both its gate structure and smaller hidden width; comparative classification accuracy and execution speed require a separate model benchmark.

### C. Rejection and Command Mapping

The decision rule is

$$c = \begin{cases} \text{map}(k), & k \neq 0,\ p_k \geq 0.80, \\ \text{NO_COMMAND}, & \text{otherwise}, \end{cases} \tag{3}$$

where $k = \arg\max_j p_j$ and class 0 is background. The threshold is fixed before testing. Background predictions produce no command even at high confidence, and uncertain gesture predictions are suppressed. Accepted commands are printed to the console. A continuous controller can extend this decision layer with persistence, cooldown, sequence numbers, and acknowledgements to manage repeated windows and message delivery.

### D. Resource and Quantization Constraints

At deployment, flash must accommodate model data, inference kernels, firmware, and constants. Peak RAM includes the ring buffer, normalized input, recurrent states, scratch arena, stack, and any radio buffers. One $64 \times 6$ window contains 768 bytes of 16-bit samples or 1536 bytes of FP32 input; these are payload calculations, not total MCU RAM. The implemented FP32 model has 4880 bytes of parameter payload, excluding host runtime and serialization overhead.

An extension to 8-bit post-training quantization would represent $r \approx s(q-z)$ using scale $s$ and zero point $z$, calibrated on training data. Deployment would require compatible integer GRU gates, nonlinearities, recurrent states, and conversion semantics. Wider biases, metadata, fallback operations, and scratch storage affect the achievable savings. Structured pruning could further reduce tensor dimensions, whereas unstructured sparsity requires supporting storage formats and kernels. Arduino Nano 33 BLE, ESP32, and LPC1768-based systems illustrate different MCU, memory, peripheral, and runtime configurations; the architecture leaves target selection to a resource and operator compatibility assessment.

## V. Software Demonstration and Pseudocode

### A. Simulation Design and Data Separation

The evaluation uses computer-generated IMU-like signals to exercise the processing and command-generation pipeline under controlled conditions. Each independent sequence contains 192 samples and six channels in nominal acceleration and angular-velocity units. Gravity is represented by an approximately constant $a_z$ offset. Background sequences contain low-amplitude noise and drift. Left and right classes use opposite-sign angular-velocity pulses with correlated acceleration, while wave sequences contain oscillatory acceleration and angular velocity. Frequency, phase, gain, offset, and noise vary independently across sequences. The generator defines repeatable motion patterns for software evaluation rather than a biomechanical model of individual users.

The random seed is 42. For each class, 24 generated sequences are assigned to training, 8 to validation, and 10 to testing *before filtering and windowing*. Each sequence yields five windows. The partitions therefore contain 480, 160, and 200 windows from 96, 32, and 40 independent sequences, respectively. Sequence identifiers and generator seeds are retained for reproducibility. Assigning complete sequences before windowing prevents overlapping samples from leaking across partitions.

### B. Training and Reproduction

Training uses FP32 CPU execution, Adam with learning rate 0.004, batch size 64, and categorical cross-entropy for 30 epochs. The checkpoint with the lowest validation cross-entropy is retained. Test labels are used only after this selection. The same fixed filtering and normalization pipeline is applied to every split. Fig. 2 summarizes the executed process; Fig. 3 presents its central logic as pseudocode.

The implementation uses Python, NumPy, and PyTorch, with the execution environment and selected checkpoint retained for reproducibility. Seven automated checks cover sequence separation, training-only normalization, causal filtering, input validation, confidence rejection, metric computation, and tensor dimensions. A separate training run reproduced the classification metrics and example command outputs.

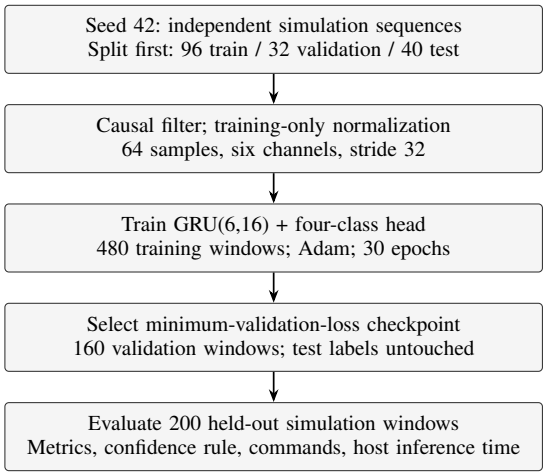



Fig. 2. Simulation-based evaluation workflow. Independent sequences are partitioned before windowing. Training data determine normalization and model weights, validation selects the checkpoint, and the held-out test set evaluates the resulting classifier and command output.

**Algorithm 1: Simulation-based command recognition**

1. Set seed 42; assign independent recording IDs to splits.
2. Generate six-axis signals for each labelled recording.
3. Causally filter each recording; reset filter at boundaries.
4. Fit normalization constants from training samples only.
5. Normalize all splits with those frozen constants.
6. Form 64-sample windows, stride 32, within each recording.
7. Train GRU; retain lowest-validation-loss checkpoint.
8. For each held-out test window:
   $p \leftarrow \mathrm{softmax}(\mathrm{GRU}(\mathrm{window}))$.
   $k \leftarrow \arg\max p$.
   If $k$ is background or $p_k < 0.80$:
      $c \leftarrow$ NO_COMMAND.
   Else: $c \leftarrow \mathrm{command_map}[k]$.
   Print label, confidence, and $c$.
9. Save metrics, split manifest, model, and example output.

Fig. 3. Pseudocode of the implemented demonstration.

## VI. Results, Discussion, and Limitations

### A. Simulation Results

Table II summarizes the recorded evaluation. Precision, recall, and F1 are computed from argmax predictions before command rejection, with macro averages assigning equal weight to all four classes. Accuracy is $\sum_k TP_k/n$, precision is $TP_k/(TP_k + FP_k)$, recall is $TP_k/(TP_k + FN_k)$, and $F1_k$ is the harmonic mean of precision and recall. Undefined denominators are assigned zero. The test partition contains $50$ windows per class.

The confusion matrix is diagonal, with $50$ correctly classified windows in each class. Example outputs are background $\rightarrow$ `NO_COMMAND`, left $\rightarrow$ `LIGHT_OFF`, right $\rightarrow$ `LIGHT_ON`, and wave $\rightarrow$ `FAN_TOGGLE`. All 150 accepted gesture-window commands match the corresponding test labels, while the background windows produce no command. An independent rule check confirms that a non-background confidence of $0.70$ is rejected by the $0.80$ threshold.

The high classification scores are consistent with the separable motion patterns and shared generation rules used across the partitions. They indicate that the processing, recurrent classification, and decision stages operate consistently within the evaluated simulation distribution. The experiment therefore establishes functional pipeline behaviour under controlled input conditions.

TABLE II
Simulation Test Results and Host Resource Measurements

| Metric | Recorded value |
|---|---|
| Held-out simulation windows | 200 (40 sequences) |
| Accuracy | 100.0% |
| Macro precision / recall / F1 | 1.000 / 1.000 / 1.000 |
| Accepted command windows | 150 of 150 gesture windows |
| Commands on background windows | 0 of 50 windows |
| Host median / p95 model latency | 0.668 / 0.949 ms |
| FP32 parameters / weight payload | 1220 / 4880 bytes |
| Saved checkpoint size | 7720 bytes |

### B. Timing and Embedded Interpretation

Model timing uses a prepared batch-one input, CPU inference mode, one PyTorch thread, ten warm-up calls, and $100$ timed calls. Table II reports the resulting host median and 95th-percentile inference times. The measurement boundary covers model execution; acquisition, window filling, preprocessing, decision logic, and communication contribute separately to end-to-end delay. Target-specific kernels and scheduling determine the corresponding MCU execution time.

The FP32 parameter payload is 4880 bytes, while the 7720-byte checkpoint additionally contains serialization information and preprocessing constants. Firmware flash and peak RAM depend on the target runtime, activation storage, stack, and concurrent buffers. On-target assessment would use a linker map, arena allocation, and stack high-water measurements. Energy evaluation would integrate supply measurements as $E = \int V(t)I(t)\,dt$, separating sensing, inference, and radio operating states.

### C. Limitations and Future Work

Evaluation is limited to simulation-generated sequences with controlled variation. Generalization to human gestures requires participant- and session-separated recordings that capture placement changes, orientation, tremor, fatigue, incomplete movements, and incidental activity. The confidence rule also requires continuous-stream assessment of false activations and duplicate commands. Real-user accuracy, MCU latency, and power consumption are therefore outside the current experimental results. Future work will combine user-centred gesture selection with representative recordings, event-level evaluation, and on-target profiling.

Further development will examine integer inference, target-specific operator support, and continuous command handling before integrating a wireless receiver and appliance interface. On-target measurements will establish the resulting firmware footprint, response time, and energy requirements.

## VII. Conclusion

This work integrates six-channel motion processing, temporal classification, and command generation within an embedded-oriented architecture. A compact GRU, training-only normalization, sequence-level separation, and a confidence-based decision rule provide a reproducible software pipeline. The simulation study demonstrates classification and command mapping under controlled conditions, while an analytical LSTM comparison quantifies the selected GRU's lower parameter count. The architecture incorporates buffering, runtime compatibility, and quantization considerations, providing a foundation for subsequent user evaluation and on-device integration.

## References


[1] Y. Zhou, H. Zhao, Y. Huang, M. Hefenbrock, T. Riedel, and M. Beigl, "TinyHAR: A Lightweight Deep Learning Model Designed for Human Activity Recognition," in *Proceedings of the International Symposium on Wearable Computers*, 2022. [Online]. Available: https://doi.org/10.1145/3544794.3558467

[2] A. Abedin, M. Ehsanpour, Q. Shi, H. Rezatofighi, and D. C. Ranasinghe, "Attend and Discriminate: Beyond the State-of-the-Art for Human Activity Recognition using Wearable Sensors," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2021. [Online]. Available: https://doi.org/10.1145/3448083

[3] S. S. Saha, S. S. Sandha, L. A. Garcia, and M. Srivastava, "TinyOdom: Hardware-Aware Efficient Neural Inertial Navigation," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2022. [Online]. Available: https://doi.org/10.1145/3534594

[4] J. Lin, W.-M. Chen, Y. Lin *et al.*, "MCUNet: Tiny Deep Learning on IoT Devices," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2007.10319

[5] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2110.15352

[6] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs," *IEEE Transactions on Computers*, 2021. [Online]. Available: https://doi.org/10.1109/TC.2021.3066883

[7] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-All: Train One Network and Specialize it for Efficient Deployment," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.1908.09791

[8] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient On-Device Learning," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2007.11622

[9] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-Device Training Under 256KB Memory," in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2206.15472

[10] C. Banbury, V. J. Reddi, P. Torelli *et al.*, "MLPerf Tiny Benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2106.07597